THE PRACTICAL DATA ENGINEERING TEXTBOOK

SQL • Python • Microsoft Azure

A 150+ page beginner-to-advanced offline learning and examination workbook

SQL: MySQL + Microsoft SQL Server
Python: Core programming, Pandas, APIs and data pipelines
Azure: ADLS Gen2, Data Factory, Databricks, Synapse, Fabric, streaming, security and operations

Includes worked examples, practical workshops, exercises, solution guides, two mock examinations, capstone architecture tasks and a 30-day revision sprint.

Edition: October 2026 | Offline PDF reference

DETAILED INDEX

PART I — SQL PRACTICE WORKSHOPS

- 1. Build a complete practice schema
- 2. Filtering and date boundaries
- 3. Join cardinality
- 4. Aggregation and business metrics
- 5. Subqueries and EXISTS
- 6. CTEs and recursive thinking
- 7. Window function patterns
- 8. Transactions and error safety
- 9. Index design
- 10. SQL Server T-SQL
- 11. MySQL practice
- 12. Data cleaning in SQL
- 13. Data warehouse queries
- 14. Query troubleshooting
- 15. SQL capstone

PART II — PYTHON PRACTICE WORKSHOPS

- 16. Build a clean project
- 17. Types and validation
- 18. Collections and algorithms
- 19. Functions and clean design
- 20. Files and safe paths
- 21. CSV and JSON pipeline
- 22. Exceptions and logging
- 23. OOP for data applications
- 24. Generators and streaming
- 25. Pandas cleaning lab
- 26. APIs and pagination
- 27. SQL connectivity
- 28. Testing data transformations
- 29. Incremental pipeline design
- 30. Python capstone

PART III — AZURE DATA ENGINEERING WORKSHOPS

- 31. Resource organization
- 32. Identity and RBAC
- 33. ADLS Gen2 layout
- 34. Storage security
- 35. ADF building blocks
- 36. ADF incremental loads

• 37. ADF operational monitoring

• 38. Databricks Spark fundamentals

• 39. Spark performance

• 40. Delta Lake and medallion

• 41. Synapse design

• 42. Fabric architecture

• 43. Streaming architecture

• 44. Governance and reliability

• 45. Azure capstone architecture

PART IV — DEEP-DIVE LESSONS

• SQL: relational logic; keys; NULL; joins; set operations; windows; transactions; indexes; statistics; dynamic SQL; MySQL; T-SQL; normalization; warehousing; SCD; profiling; tuning; recovery; security; capstone.

• Python: object model; numeric precision; complexity; function contracts; iterators; exceptions; file encoding; JSON; Pandas indexing; merge; missing data; performance; testing; logging; APIs; database transactions; concurrency; idempotency; capstone.

• Azure: resource hierarchy; identity; RBAC; storage redundancy; ADLS ACLs; file formats; ADF; watermarking; expressions; runtimes; Spark; optimization; Delta; Synapse; Fabric; streaming; security; capstone and operations.

PART V — EXAM PREPARATION AND REVISION

• SQL intermediate assessment and answer guide

• Python intermediate assessment and answer guide

• Azure scenario assessment and answer guide

• Full mock examination A

• Full mock examination B

• Glossary and final revision checklist

• 12-week study plan

• 30-day final revision sprint

How to study: read each deep-dive lesson, run or adapt the example, answer the review questions, then complete the capstone and mock exams. Always verify current Azure features and exam objectives in official Microsoft Learn documentation when online.

SQL Workshop 1 — Build a complete practice schema

Create a small sales database and understand entity relationships.

Concepts to understand

• Identify primary keys and foreign keys.

• Use exact numeric types for prices.

• Make required fields NOT NULL.

• Use a consistent naming convention.

Worked example

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  City VARCHAR(60)
);
CREATE TABLE SalesOrder (
  OrderID INT PRIMARY KEY,
  CustomerID INT NOT NULL,
  OrderDate DATE NOT NULL,
  Status VARCHAR(20) NOT NULL,
  FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
);
```

Your practice tasks

1. Add a Product table with a unique product code.
2. Add an OrderLine table with quantity and unit price.
3. Explain why OrderLine needs its own grain.
4. Insert five realistic but fictional records.

Self-check / solution outline

The OrderLine grain should be one product line within one order. A composite key such as (OrderID, LineNumber) or a surrogate OrderLineID can identify it.

SQL Workshop 2 — Filtering and date boundaries

Practise correct date filtering, NULL handling and search conditions.

Concepts to understand

• Use half-open date ranges for datetime columns.

• Use IS NULL for missing values.

• Use parentheses around mixed AND/OR conditions.

• Avoid applying YEAR() to an indexed date column.

Worked example

```
SELECT OrderID, OrderDate, Status
FROM SalesOrder
WHERE OrderDate >= '2026-01-01'
  AND OrderDate <  '2026-02-01'
  AND (Status = 'Open' OR Status = 'Processing')
ORDER BY OrderDate, OrderID;
```

Your practice tasks

1. Find orders in the last full calendar month using explicit boundaries.
2. Find rows where City is missing.
3. Find customer names beginning with S.
4. Explain why BETWEEN can be risky for datetime end dates.

Self-check / solution outline

A half-open interval includes the start and excludes the next period's start, avoiding missing times later on the end date.

SQL Workshop 3 — Join cardinality

Understand one-to-one, one-to-many and many-to-many relationships.

Concepts to understand

• One-to-many joins can multiply rows.

• Many-to-many relationships need a bridge table.

• Check uniqueness on the dimension side.

• Aggregate at the correct grain.

Worked example

```
SELECT c.CustomerID, c.CustomerName, o.OrderID
FROM Customer AS c
LEFT JOIN SalesOrder AS o
  ON o.CustomerID = c.CustomerID;
```

Your practice tasks

1. Count customers before and after the join.
2. Find customers with no orders.
3. Create a CustomerTag bridge table.
4. Explain why joining two one-to-many tables can multiply rows.

Self-check / solution outline

When both child tables have multiple rows per parent, joining them together can create a cross product within each parent. Aggregate each child to parent grain before combining.

SQL Workshop 4 — Aggregation and business metrics

Build useful sales measures and distinguish row counts from distinct entity counts.

Concepts to understand

• COUNT(*) counts rows.

• COUNT(DISTINCT key) counts unique keys.

• SUM ignores NULL values.

• Define metric grain and business meaning.

Worked example

```
SELECT CustomerID,
       COUNT(*) AS order_count,
       COUNT(DISTINCT Status) AS distinct_statuses
FROM SalesOrder
GROUP BY CustomerID;
```

Your practice tasks

1. Calculate average order value.
2. Calculate cancelled-order percentage.
3. Report monthly order counts.
4. Explain how NULL amounts affect AVG.

Self-check / solution outline

For a percentage, guard against a zero denominator with NULLIF and document whether the denominator is all orders or only valid orders.

SQL Workshop 5 — Subqueries and EXISTS

Use subqueries to express existence and comparison logic.

Concepts to understand

• EXISTS is true when at least one row matches.

• NOT EXISTS is a clear anti-join.

• Correlated subqueries refer to outer query columns.

• Check NULL behavior with NOT IN.

Worked example

```
SELECT c.CustomerID, c.CustomerName
FROM Customer AS c
WHERE NOT EXISTS (
  SELECT 1
  FROM SalesOrder AS o
  WHERE o.CustomerID = c.CustomerID
);
```

Your practice tasks

1. Find customers with at least one order.
2. Find customers with no orders.
3. Find orders above the average order amount.
4. Explain the NULL trap with NOT IN.

Self-check / solution outline

NOT IN can behave unexpectedly if the subquery contains NULL. NOT EXISTS usually expresses anti-join intent more safely.

SQL Workshop 6 — CTEs and recursive thinking

Use CTEs to make multi-step logic readable.

Concepts to understand

• A CTE names a query expression.

• Multiple CTEs can form readable stages.

• Recursive CTEs traverse hierarchies.

• CTEs are not automatically materialized.

Worked example

```
WITH CustomerOrderCounts AS (
  SELECT CustomerID, COUNT(*) AS OrderCount
  FROM SalesOrder
  GROUP BY CustomerID
)
SELECT c.CustomerName, COALESCE(x.OrderCount, 0) AS OrderCount
FROM Customer c
LEFT JOIN CustomerOrderCounts x
  ON x.CustomerID = c.CustomerID;
```

Your practice tasks

1. Create a CTE for monthly sales.
2. Create a second CTE for a moving total.
3. Describe a manager hierarchy recursive CTE.
4. Explain why CTE syntax does not guarantee a performance improvement.

Self-check / solution outline

A CTE primarily improves expression and maintainability; the optimizer decides the physical execution strategy.

SQL Workshop 7 — Window function patterns

Use ranking, offsets and running calculations without collapsing rows.

Concepts to understand

• PARTITION BY defines independent groups.

• ORDER BY defines sequence within a window.

• ROW_NUMBER breaks ties deterministically with a tie-breaker.

• LAG and LEAD compare neighboring rows.

Worked example

```
SELECT CustomerID, OrderDate, OrderID,
       ROW_NUMBER() OVER (
         PARTITION BY CustomerID
         ORDER BY OrderDate, OrderID
       ) AS sequence_no,
       LAG(OrderDate) OVER (
         PARTITION BY CustomerID
         ORDER BY OrderDate, OrderID
       ) AS previous_order_date
FROM SalesOrder;
```

Your practice tasks

1. Return each customer's first order.
2. Calculate days since previous order.
3. Rank orders by value per customer.
4. Explain the difference between RANK and DENSE_RANK.

Self-check / solution outline

Always use a deterministic tie-breaker when the sequence must be stable.

SQL Workshop 8 — Transactions and error safety

Practise atomic multi-statement changes and safe rollback.

Concepts to understand

• Keep transactions short.

• Validate affected row counts.

• Use TRY/CATCH in SQL Server where appropriate.

• Know the storage engine and isolation behavior.

Worked example

```
-- Generic pattern; adapt error handling to the engine
START TRANSACTION;
UPDATE Account
SET Balance = Balance - 100
WHERE AccountID = 1 AND Balance >= 100;
-- Check that exactly one row was updated.
UPDATE Account
SET Balance = Balance + 100
WHERE AccountID = 2;
COMMIT;
-- ROLLBACK if validation or any step fails.
```

Your practice tasks

1. Add a sufficient-balance check.
2. Explain why checking row count matters.
3. Describe a deadlock and a safe retry strategy.
4. Explain why external API calls should not be held inside long DB transactions.

Self-check / solution outline

A transaction must roll back if the debit did not affect exactly one intended row. Production code should use engine-appropriate exception handling.

SQL Workshop 9 — Index design

Choose indexes based on workload evidence.

Concepts to understand

• Index predicates and join keys that matter.

• Composite index leading columns matter.

• Indexes add write and storage overhead.

• Review actual execution plans and statistics.

Worked example

```
CREATE INDEX IX_SalesOrder_Customer_Date
ON SalesOrder(CustomerID, OrderDate);
```

Your practice tasks

1. Which query benefits from this index?
2. Would a date-only query necessarily use it efficiently?
3. Design a covering index for a report and explain trade-offs.
4. List signs that an index is unused or harmful.

Self-check / solution outline

The index naturally supports filtering by CustomerID and then OrderDate. It is not equivalent to an index beginning with OrderDate.

SQL Workshop 10 — SQL Server T-SQL

Practise common SQL Server-specific constructs.

Concepts to understand

• Use IDENTITY for generated numeric keys.

• Use TOP or OFFSET/FETCH for paging.

• Use TRY/CATCH for error handling.

• GO is a client batch separator.

Worked example

```
BEGIN TRY
  BEGIN TRANSACTION;
  UPDATE dbo.Employee
  SET Salary = Salary * 1.05
  WHERE DepartmentID = 4;
  COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
  THROW;
END CATCH;
```

Your practice tasks

1. Create a temporary table.
2. Write a stored procedure with an input parameter.
3. Explain SET NOCOUNT ON.
4. Explain why dynamic SQL needs parameterization.

Self-check / solution outline

T-SQL stored procedures can centralize logic, but application inputs must still be parameterized and permissions tightly controlled.

SQL Workshop 11 — MySQL practice

Practise MySQL-specific schema and query patterns.

Concepts to understand

• Use AUTO_INCREMENT where appropriate.

• InnoDB supports transactions and foreign keys.

• Use LIMIT for result limiting.

• Understand SQL mode and collation.

Worked example

```
CREATE TABLE Asset (
  AssetID INT AUTO_INCREMENT PRIMARY KEY,
  AssetTag VARCHAR(40) NOT NULL UNIQUE,
  PurchaseCost DECIMAL(12,2),
  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Your practice tasks

1. Add a foreign key to an AssetCategory table.
2. Write an upsert using ON DUPLICATE KEY UPDATE.
3. Explain why a timestamp default is useful.
4. Explain the role of ONLY_FULL_GROUP_BY.

Self-check / solution outline

The unique asset tag prevents duplicates. Exact upsert behavior should be checked against the target MySQL version.

SQL Workshop 12 — Data cleaning in SQL

Use SQL to profile and standardize messy source data.

Concepts to understand

• Trim leading and trailing spaces.

• Normalize case only where business rules allow.

• Convert safely and count invalid values.

• Keep rejected records for remediation.

Worked example

```
SELECT
  COUNT(*) AS total_rows,
  SUM(CASE WHEN Email IS NULL OR TRIM(Email) = '' THEN 1 ELSE 0 END) AS missing_email
FROM StagingEmployee;
```

Your practice tasks

1. Find duplicate emails.
2. Identify malformed date strings before conversion.
3. Create a rejected-record reason column.
4. Explain why deleting all duplicates may lose valid history.

Self-check / solution outline

Profile first, define the business key, rank duplicates deterministically, then quarantine or resolve duplicates according to an approved rule.

SQL Workshop 13 — Data warehouse queries

Build a simple star-schema query.

Concepts to understand

• Fact grain should be explicit.

• Dimensions describe facts.

• Surrogate keys support history.

• Date dimensions simplify calendar reporting.

Worked example

```
SELECT d.CalendarYear, p.Category,
       SUM(f.SalesAmount) AS Revenue
FROM FactSales f
JOIN DimDate d ON d.DateKey = f.DateKey
JOIN DimProduct p ON p.ProductKey = f.ProductKey
GROUP BY d.CalendarYear, p.Category;
```

Your practice tasks

1. State the fact grain.
2. Add a customer dimension.
3. Explain SCD Type 1 and Type 2.
4. Describe why a date dimension can be better than ad hoc date extraction.

Self-check / solution outline

The query assumes one fact row per defined sales event and valid dimension keys. Document whether returns and tax are included in Revenue.

SQL Workshop 14 — Query troubleshooting

Use a repeatable process for slow or incorrect queries.

Concepts to understand

• Reproduce with representative data.

• Inspect actual plan and row estimates.

• Check data skew and join cardinality.

• Change one thing at a time and measure.

Worked example

```
-- First inspect the query and row counts.
SELECT COUNT(*) FROM SalesOrder;
SELECT CustomerID, COUNT(*) AS n
FROM SalesOrder
GROUP BY CustomerID
ORDER BY n DESC;
```

Your practice tasks

1. Find the largest customer groups.
2. Check whether a predicate is SARGable.
3. Explain why SELECT * can be wasteful.
4. Write a short tuning note with baseline and new runtime.

Self-check / solution outline

A tuning note should include query, data volume, plan observations, change made, before/after timing and correctness checks.

SQL Workshop 15 — SQL capstone

Combine schema, queries, quality checks and operational thinking.

Concepts to understand

• Design a small asset inventory schema.

• Use keys and constraints.

• Create a report with joins and window functions.

• Protect changes with transactions.

Worked example

```
-- Example asset report
SELECT a.AssetTag, c.CategoryName, e.EmployeeName,
       ROW_NUMBER() OVER (
         PARTITION BY c.CategoryName
         ORDER BY a.PurchaseDate, a.AssetID
       ) AS category_sequence
FROM Asset a
JOIN AssetCategory c ON c.CategoryID = a.CategoryID
LEFT JOIN Employee e ON e.EmployeeID = a.AssignedEmployeeID;
```

Your practice tasks

1. Add assignment history.
2. Report assets without an assigned employee.
3. Find duplicate serial numbers.
4. Write a transaction for changing an asset's assignment.
5. Document data-quality and security controls.

Self-check / solution outline

A robust design separates Asset from AssetAssignment so assignment history is preserved rather than overwritten.

Python Workshop 1 — Build a clean project

Set up a repeatable local Python project for offline development.

Concepts to understand

• Use a virtual environment.

• Separate source, tests and data folders.

• Pin dependencies for reproducibility.

• Keep secrets outside source control.

Worked example

```
project/
  src/
    pipeline.py
  tests/
    test_pipeline.py
  data/
  requirements.txt
  README.md
```

Your practice tasks

1. Create and activate a virtual environment.
2. Install Pandas in the environment.
3. Create a requirements file.
4. Write a README with run instructions.

Self-check / solution outline

The environment isolates dependencies. A README should record Python version, setup, execution, inputs, outputs and troubleshooting.

Python Workshop 2 — Types and validation

Validate incoming records before transforming them.

Concepts to understand

• Convert external text at the boundary.

• Catch ValueError and TypeError.

• Distinguish missing from invalid.

• Return actionable validation messages.

Worked example

```
def parse_quantity(raw):
    if raw is None or str(raw).strip() == "":
        raise ValueError("quantity is required")
    try:
        value = int(raw)
    except (TypeError, ValueError) as exc:
        raise ValueError("quantity must be an integer") from exc
    if value < 0:
        raise ValueError("quantity cannot be negative")
    return value
```

Your practice tasks

1. Extend validation to positive unit prices.
2. Reject boolean values if they are not valid quantities.
3. Test zero, negative, blank and malformed input.
4. Explain why validation belongs near ingestion.

Self-check / solution outline

Validation near ingestion prevents bad values from spreading and makes rejected records traceable.

Python Workshop 3 — Collections and algorithms

Choose collections and understand time complexity.

Concepts to understand

• Dictionary for key lookup.

• Set for uniqueness and membership.

• List for ordered accumulation.

• Avoid repeated linear searches in large lists.

Worked example

```
def deduplicate(rows, key):
    seen = set()
    result = []
    for row in rows:
        value = row[key]
        if value not in seen:
            seen.add(value)
            result.append(row)
    return result
```

Your practice tasks

1. Deduplicate employees by employee_id.
2. Count records by department using a dict.
3. Find values present in one source but not another.
4. Explain average O(1) set membership.

Self-check / solution outline

This approach preserves first-seen order and uses a set for fast membership. The key values must be hashable.

Python Workshop 4 — Functions and clean design

Refactor repeated logic into small reusable functions.

Concepts to understand

• Single responsibility improves testability.

• Prefer return values over printing inside logic.

• Use explicit arguments.

• Avoid hidden global state.

Worked example

```
def calculate_line_total(quantity, unit_price):
    if quantity < 0:
        raise ValueError("quantity cannot be negative")
    if unit_price < 0:
        raise ValueError("unit price cannot be negative")
    return quantity * unit_price
```

Your practice tasks

1. Write a discount function.
2. Write a function that normalizes a phone string without assuming country rules.
3. Add docstrings and type hints.
4. Write tests for boundaries.

Self-check / solution outline

A function should define assumptions, inputs, output and failure behavior. Type hints document intent but do not validate at runtime by themselves.

Python Workshop 5 — Files and safe paths

Read and write files without loading unnecessary data.

Concepts to understand

• Use pathlib for paths.

• Use context managers.

• Specify encoding.

• Stream large files line by line.

Worked example

```
from pathlib import Path

source = Path("input.txt")
with source.open(encoding="utf-8") as handle:
    for line in handle:
        process(line.rstrip("\n"))
```

Your practice tasks

1. Handle a missing input file.
2. Write output to a temporary path first.
3. Count lines without storing them all.
4. Explain the difference between append and overwrite.

Self-check / solution outline

Streaming line-by-line keeps memory bounded for many text processing tasks.

Python Workshop 6 — CSV and JSON pipeline

Convert source files into validated structured records.

Concepts to understand

• Use csv.DictReader for CSV.

• Use json.load for a JSON file.

• Validate required fields.

• Preserve source filename and row number for rejected data.

Worked example

```
import csv

accepted, rejected = [], []
with open("assets.csv", newline="", encoding="utf-8") as f:
    for line_no, row in enumerate(csv.DictReader(f), start=2):
        if not row.get("asset_tag", "").strip():
            rejected.append((line_no, row, "missing asset_tag"))
        else:
            accepted.append(row)
```

Your practice tasks

1. Add duplicate tag detection.
2. Parse a purchase date.
3. Write accepted and rejected output files.
4. Explain how you would handle malformed CSV rows.

Self-check / solution outline

A good pipeline keeps a rejection file with row number and reason, enabling remediation without silently dropping source data.

Python Workshop 7 — Exceptions and logging

Build observable scripts that fail clearly.

Concepts to understand

• Catch specific exceptions.

• Log context, not secrets.

• Re-raise when a caller needs to know.

• Use severity levels consistently.

Worked example

```
import logging
logger = logging.getLogger(__name__)

def load_config(path):
    try:
        return read_config(path)
    except FileNotFoundError:
        logger.exception("Configuration file is missing: %s", path)
        raise
```

Your practice tasks

1. Add logging for start, success and failure.
2. Create a custom validation exception.
3. Explain when to use warning versus error.
4. Ensure no password appears in logs.

Self-check / solution outline

Logs should explain what happened, where, and with which run identifier while excluding secrets and unnecessary personal information.

Python Workshop 8 — OOP for data applications

Model a simple asset domain without overengineering.

Concepts to understand

• Use classes when state and behavior belong together.

• Use dataclasses for simple records.

• Prefer composition for flexible relationships.

• Test public behavior.

Worked example

```
from dataclasses import dataclass

@dataclass
class Asset:
    asset_tag: str
    category: str
    status: str = "In Stock"

    def assign(self):
        if self.status != "In Stock":
            raise ValueError("Asset is not available")
        self.status = "Assigned"
```

Your practice tasks

1. Add an Employee class.
2. Represent assignment as a separate object.
3. Explain why assignment history should not be stored only as current status.
4. Write tests for invalid assignment.

Self-check / solution outline

Separating Assignment from Asset supports history, dates, handover and auditability.

Python Workshop 9 — Generators and streaming

Process large inputs lazily.

Concepts to understand

• Generators yield one item at a time.

• Avoid collect-like materialization for large inputs.

• Pipeline stages can be chained.

• Close resources deterministically.

Worked example

```
def valid_rows(rows):
    for row in rows:
        if row.get("id") and row.get("amount") is not None:
            yield row

def amounts(rows):
    for row in valid_rows(rows):
        yield float(row["amount"])
```

Your practice tasks

1. Build a generator that filters dates.
2. Calculate a running total from a generator.
3. Explain what happens if a generator is consumed twice.
4. Compare a list comprehension and generator expression.

Self-check / solution outline

A generator is an iterator and is normally exhausted after one pass. Create a new generator when another pass is required.

Python Workshop 10 — Pandas cleaning lab

Clean a tabular dataset using vectorized operations.

Concepts to understand

• Inspect before modifying.

• Convert types explicitly.

• Track invalid values.

• Use merge validation.

Worked example

```
import pandas as pd

df = pd.read_csv("employees.csv")
df["employee_id"] = pd.to_numeric(df["employee_id"], errors="coerce")
df["joining_date"] = pd.to_datetime(df["joining_date"], errors="coerce")
df["name"] = df["name"].astype("string").str.strip()
invalid = df[df["employee_id"].isna() | df["joining_date"].isna()]
```

Your practice tasks

1. Report null count by column.
2. Remove duplicate IDs only after defining the rule.
3. Group headcount by department.
4. Merge with a department reference and identify unmatched IDs.

Self-check / solution outline

Use invalid rows as a separate output and record why they failed; do not discard them without review.

Python Workshop 11 — APIs and pagination

Retrieve all pages from an API safely.

Concepts to understand

• Set request timeouts.

• Check status codes.

• Handle rate limits and retries.

• Protect tokens and follow pagination links carefully.

Worked example

```
# Conceptual pagination loop
items = []
next_url = initial_url
while next_url:
    response = requests.get(next_url, timeout=20)
    response.raise_for_status()
    payload = response.json()
    items.extend(payload["items"])
    next_url = payload.get("next")
```

Your practice tasks

1. Add a maximum page count safeguard.
2. Avoid retrying permanent 4xx errors.
3. Record page counts and total rows.
4. Explain why API pagination can change while data is being updated.

Self-check / solution outline

For changing source data, use snapshot tokens or stable cursor pagination where the API provides them.

Python Workshop 12 — SQL connectivity

Write safe database access code.

Concepts to understand

• Use parameterized SQL.

• Use transactions for multi-step writes.

• Close connections reliably.

• Avoid embedding passwords.

Worked example

```
# Illustrative DB-API pattern
with connection.cursor() as cursor:
    cursor.execute(
        "SELECT asset_id FROM asset WHERE asset_tag = %s",
        (asset_tag,)
    )
    result = cursor.fetchone()
```

Your practice tasks

1. Write an insert using parameters.
2. Explain placeholder differences across drivers.
3. Handle connection timeout.
4. Describe how a secret manager can supply credentials.

Self-check / solution outline

Parameter placeholder style is driver-specific. Confirm the DB-API driver's documentation and never concatenate user input into SQL.

Python Workshop 13 — Testing data transformations

Build tests that protect business rules.

Concepts to understand

• Test typical and edge cases.

• Test invalid inputs.

• Use small deterministic fixtures.

• Test idempotency where relevant.

Worked example

```
def normalize_tag(value):
    if value is None:
        raise ValueError("tag required")
    tag = str(value).strip().upper()
    if not tag:
        raise ValueError("tag required")
    return tag

def test_normalize_tag():
    assert normalize_tag(" bel-1 ") == "BEL-1"
```

Your practice tasks

1. Test None and empty strings.
2. Test leading/trailing whitespace.
3. Test duplicate handling.
4. Add a test for malformed date conversion.

Self-check / solution outline

Tests should be small, fast and deterministic. A passing test suite is evidence, not proof, of correctness.

Python Workshop 14 — Incremental pipeline design

Build a restartable and idempotent batch pipeline.

Concepts to understand

• Use a stable business key.

• Separate staging from publication.

• Checkpoint only after successful output.

• Make retries safe.

Worked example

```
def process_batch(rows, existing_by_id):
    staged = {}
    for row in rows:
        key = row["record_id"]
        staged[key] = transform(row)
    # Publish using an upsert/merge operation.
    return staged
```

Your practice tasks

1. Define a watermark field.
2. Handle deletes or tombstones.
3. Explain late-arriving updates.
4. Describe how to recover after publish succeeds but checkpoint update fails.

Self-check / solution outline

If publication succeeds but checkpoint update fails, rerunning must safely merge the same keys without duplicating rows.

Python Workshop 15 — Python capstone

Create a mini pipeline that reads, validates, transforms and reports.

Concepts to understand

• Separate extraction, validation, transformation and load.

• Record row counts at each stage.

• Keep rejected rows with reasons.

• Use configuration and structured logs.

Worked example

```
def run_pipeline(source_rows):
    valid, rejected = validate_rows(source_rows)
    transformed = [transform(row) for row in valid]
    load_rows(transformed)
    return {
        "source_count": len(source_rows),
        "accepted_count": len(transformed),
        "rejected_count": len(rejected)
    }
```

Your practice tasks

1. Add tests for each stage.
2. Add a run ID to logs.
3. Make load idempotent.
4. Write a README describing inputs, outputs and recovery.

Self-check / solution outline

A capstone is complete when it can be rerun safely, reconciles counts, exposes rejected rows and documents recovery.

Azure Workshop 1 — Resource organization

Plan a clean Azure environment before deploying services.

Concepts to understand

• Separate dev, test and production.

• Use resource groups by lifecycle and ownership.

• Apply tags for cost allocation.

• Use least privilege.

Worked example

```
Example naming pattern:
rg-data-dev-centralindia
stcompanydatalake01
adf-company-ingest-dev
kv-company-data-dev
```

Your practice tasks

1. Design resource groups for a small data platform.
2. Define tags for owner, environment and cost center.
3. Choose a region based on residency and service availability.
4. Explain why a resource group is not a security boundary by itself.

Self-check / solution outline

Resource groups organize lifecycle and access scope, but networking, identity and data-plane controls still need explicit design.

Azure Workshop 2 — Identity and RBAC

Grant services the minimum access needed.

Concepts to understand

• Use managed identity when supported.

• Assign roles at the narrowest practical scope.

• Separate control-plane and data-plane access.

• Review access periodically.

Worked example

```
Design pattern:
Data Factory managed identity
  -> Storage Blob Data Contributor on target filesystem
  -> Key Vault Secrets User only if secrets are required
Databricks identity
  -> scoped access to approved data locations
```

Your practice tasks

1. Explain a system-assigned versus user-assigned managed identity.
2. Give an example of least-privilege storage access.
3. Explain why Owner is too broad for routine pipeline execution.
4. Describe an access review.

Self-check / solution outline

A pipeline should not be Owner of a subscription merely to write files to one approved storage location.

Azure Workshop 3 — ADLS Gen2 layout

Organize lake paths for maintainability and query performance.

Concepts to understand

• Use landing/raw/bronze/silver/gold intentionally.

• Partition by useful query dimensions.

• Avoid too many tiny files.

• Define retention and ownership.

Worked example

```
/landing/erp/orders/ingest_date=2026-10-31/
/bronze/erp/orders/
/silver/sales/orders/
/gold/sales/monthly_revenue/
```

Your practice tasks

1. Design paths for daily customer extracts.
2. Choose whether customer_id is a good partition key.
3. Explain small-file problems.
4. Define lifecycle rules for raw files.

Self-check / solution outline

High-cardinality partitions such as customer_id can create many small folders. Date partitions are often more manageable for batch analytics.

Azure Workshop 4 — Storage security

Protect storage accounts and data paths.

Concepts to understand

• Disable anonymous access unless explicitly needed.

• Use Entra ID and scoped roles.

• Use private endpoints where appropriate.

• Protect keys and SAS tokens.

Worked example

```
Security review:
- Public network access: evaluate and restrict
- Shared key access: disable if compatible with workload
- Private endpoint and DNS: test resolution
- RBAC/ACL: least privilege
- Diagnostics: enable appropriate logs
```

Your practice tasks

1. Explain RBAC and ACL differences.
2. Describe private endpoint DNS requirements.
3. Explain why a leaked SAS token is dangerous.
4. List audit events worth monitoring.

Self-check / solution outline

A private endpoint alone does not grant data permission; identity and authorization are still required.

Azure Workshop 5 — ADF building blocks

Map a business workflow to ADF components.

Concepts to understand

• Linked service describes connection.

• Dataset describes data structure/location.

• Pipeline orchestrates activities.

• Integration Runtime supplies connectivity/compute.

Worked example

```
Pipeline: DailyOrders
  1. Lookup previous watermark
  2. Copy changed source rows to staging
  3. Validate row count and schema
  4. Execute merge or notebook
  5. Update watermark
  6. Send success notification
```

Your practice tasks

1. Identify the linked services and datasets.
2. Choose an integration runtime for on-premises SQL Server.
3. Parameterize the source table.
4. Describe failure and retry paths.

Self-check / solution outline

Use a self-hosted integration runtime when the cloud service needs a managed bridge to a private on-premises network.

Azure Workshop 6 — ADF incremental loads

Design a watermark load that can recover from failures.

Concepts to understand

• Watermark is committed only after successful publish.

• Use overlap windows if late updates are possible.

• Handle deletes separately.

• Make target writes idempotent.

Worked example

```
Concept:
old_watermark = read_control_table()
batch = source where modified_at > old_watermark
stage(batch)
validate(batch)
merge_stage_to_target()
new_watermark = max(batch.modified_at)
write_control_table(new_watermark)
```

Your practice tasks

1. What if the batch is empty?
2. What if the merge succeeds but watermark update fails?
3. What if two rows share the same timestamp?
4. How would CDC change the design?

Self-check / solution outline

A composite watermark (timestamp plus stable key) or source CDC can avoid ambiguity when timestamps are not unique.

Azure Workshop 7 — ADF operational monitoring

Make pipelines observable and supportable.

Concepts to understand

• Capture run and activity IDs.

• Alert on actionable failures.

• Record row counts and duration.

• Distinguish transient and permanent errors.

Worked example

```
Operational record fields:
pipeline_name
run_id
source_window_start
source_window_end
source_count
target_count
rejected_count
duration_seconds
status
error_category
```

Your practice tasks

1. Design an alert for repeated failures.
2. Describe how to rerun one failed window.
3. Explain why retrying schema errors is wasteful.
4. Create a reconciliation rule.

Self-check / solution outline

A rerun should use the same logical source window and be safe because the target operation is idempotent.

Azure Workshop 8 — Databricks Spark fundamentals

Understand the distributed execution model.

Concepts to understand

• Driver coordinates execution.

• Executors run tasks.

• Transformations are lazy.

• Actions trigger execution.

Worked example

```
from pyspark.sql import functions as F

df = spark.read.parquet("/mnt/bronze/orders")
result = (df.filter(F.col("status") == "COMPLETE")
            .groupBy("region")
            .agg(F.sum("amount").alias("revenue")))
result.write.format("delta").mode("overwrite").save("/mnt/gold/revenue")
```

Your practice tasks

1. Explain why collect() can be dangerous.
2. Identify transformations and actions.
3. Explain partitions and tasks.
4. Describe a shuffle caused by groupBy.

Self-check / solution outline

collect transfers data to the driver and can exhaust driver memory for large datasets. Prefer distributed writes or bounded samples.

Azure Workshop 9 — Spark performance

Diagnose a slow distributed job.

Concepts to understand

• Inspect Spark UI and stage metrics.

• Look for skew and shuffle spill.

• Avoid unnecessary repartitioning.

• Choose file size and cluster size deliberately.

Worked example

```
# Inspect plan and schema
result.explain("formatted")
print(result.rdd.getNumPartitions())
```

Your practice tasks

1. Explain data skew.
2. Compare repartition and coalesce conceptually.
3. Describe broadcast join use cases.
4. List metrics you would inspect in a slow stage.

Self-check / solution outline

A broadcast join can help when one side is small enough to distribute safely; validate size and memory impact.

Azure Workshop 10 — Delta Lake and medallion

Create reliable lakehouse tables.

Concepts to understand

• Delta transaction log supports table operations.

• Bronze preserves source traceability.

• Silver standardizes and validates.

• Gold serves business questions.

Worked example

```
# Illustrative Spark write
df.write.format("delta").mode("append").saveAsTable("silver.orders")

# Illustrative SQL
# SELECT COUNT(*) FROM silver.orders
```

Your practice tasks

1. Define a schema evolution policy.
2. Explain time travel use cases.
3. Describe compaction and file sizing.
4. Design a gold monthly sales table.

Self-check / solution outline

Schema evolution should be controlled; automatically accepting every new column can hide source contract changes.

Azure Workshop 11 — Synapse design

Choose the right Synapse SQL approach.

Concepts to understand

• Dedicated SQL pool is a distributed warehouse.

• Serverless SQL queries files without a dedicated provisioned pool.

• Distribution affects data movement.

• Stage bulk loads.

Worked example

```
Design comparison:
- Serverless SQL: exploratory lake queries
- Dedicated SQL pool: provisioned MPP warehouse
- Spark pool: distributed notebook processing
```

Your practice tasks

1. Explain hash, round-robin and replicated distributions.
2. Choose a distribution for a large fact table.
3. Explain why small dimensions may be replicated.
4. List workload signals for a dedicated warehouse.

Self-check / solution outline

Distribution choices depend on table size, join patterns, skew and workload. Validate with representative queries.

Azure Workshop 12 — Fabric architecture

Map analytics work to Fabric items.

Concepts to understand

• Workspace organizes collaboration and items.

• OneLake is the unified logical data lake.

• Lakehouse supports files, tables and Spark.

• Warehouse offers a SQL analytics experience.

Worked example

```
Example:
Workspace: SalesAnalytics
  Lakehouse: SalesLakehouse
  Notebook: CleanOrders
  Pipeline: DailySalesLoad
  Warehouse: SalesWarehouse
  Semantic model: SalesModel
```

Your practice tasks

1. Choose Lakehouse versus Warehouse for a use case.
2. Explain a OneLake shortcut.
3. Describe capacity monitoring.
4. Explain how a semantic model supports business reporting.

Self-check / solution outline

Choose based on engineering pattern, SQL needs, data format, governance and capacity economics; do not select only by product name.

Azure Workshop 13 — Streaming architecture

Design event ingestion with replay and duplicate handling.

Concepts to understand

• Event Hubs partitions provide parallelism.

• Ordering is within a partition.

• Consumer groups isolate reader applications.

• Event IDs support deduplication.

Worked example

```
Event producer
  -> Event Hubs
  -> Consumer group: fraud-monitor
  -> Stream processor
  -> Curated sink and alert channel
```

Your practice tasks

1. Choose a partition key.
2. Explain consumer lag.
3. Define late-event policy.
4. Describe replay and poison-event handling.

Self-check / solution outline

A stable partition key can preserve related-event ordering, but poor key choice can create partition skew.

Azure Workshop 14 — Governance and reliability

Design controls that make a data platform trustworthy.

Concepts to understand

• Catalog assets and owners.

• Track lineage and sensitivity.

• Define retention and deletion.

• Test restore and recovery.

Worked example

```
Dataset contract:
name: curated_sales
owner: Finance Analytics
grain: one row per order line
freshness_target: daily by 07:00
sensitivity: internal
quality_rules: unique order_line_id; valid date; nonnegative quantity
```

Your practice tasks

1. Define an owner and steward.
2. Write three data quality rules.
3. Define a recovery point objective.
4. Explain how lineage helps change impact analysis.

Self-check / solution outline

A dataset contract makes expectations explicit and helps teams detect breaking changes before consumers are affected.

Azure Workshop 15 — Azure capstone architecture

Design an end-to-end, secure and observable analytics platform.

Concepts to understand

• Choose services based on latency and workload.

• Use identity-based access and private networking.

• Make loads incremental and idempotent.

• Monitor quality, reliability and cost.

Worked example

```
Sources: SQL Server + daily CSV + event stream
Ingestion: ADF + Event Hubs
Lake: ADLS Gen2 / OneLake
Processing: Databricks or Fabric Spark
Serving: Synapse/Fabric Warehouse
Governance: Entra ID + RBAC/ACL + catalog
Operations: logs + alerts + reconciliation
```

Your practice tasks

1. Draw the architecture and trust boundaries.
2. Define bronze/silver/gold ownership.
3. Describe one failure and recovery path.
4. Estimate cost drivers.
5. Present a 3-minute design explanation.

Self-check / solution outline

A strong design explains why each service exists, how identities access data, how data is validated, how retries avoid duplication, and how cost and failure are controlled.

30-day final revision sprint

Use this sprint after completing the chapters and workshops. Spend 60–90 minutes per day if possible. If work is busy, complete one task at a time rather than skipping the review.

Day 1: SQL SELECT, WHERE, ORDER BY; write 15 queries.

Day 2: SQL NULL, CASE, date ranges and string functions.

Day 3: SQL GROUP BY, HAVING and conditional aggregation.

Day 4: SQL INNER/LEFT joins and anti-joins.

Day 5: SQL subqueries, EXISTS and CTEs.

Day 6: SQL window functions and ranking.

Day 7: SQL keys, constraints and normalization.

Day 8: SQL transactions, isolation and safe DML.

Day 9: SQL indexes and execution plans.

Day 10: Complete SQL mock and review errors.

Day 11: Python variables, types and operators.

Day 12: Python strings, lists, tuples, sets and dicts.

Day 13: Python conditions, loops and comprehensions.

Day 14: Python functions, modules and scope.

Day 15: Python exceptions, files, CSV and JSON.

Day 16: Python OOP, iterators and generators.

Day 17: Python testing, logging and virtual environments.

Day 18: Pandas cleaning, grouping and merging.

Day 19: API and SQL connectivity patterns.

Day 20: Build a small Python ETL project.

Day 21: Azure cloud, resource hierarchy and identity.

Day 22: ADLS Gen2, zones, formats and security.

Day 23: ADF components, Copy activity and runtime.

Day 24: Watermarks, triggers and pipeline monitoring.

Day 25: Databricks, Spark DataFrames and shuffles.

Day 26: Delta Lake and medallion architecture.

Day 27: Synapse and Fabric comparison.

Day 28: Streaming, Event Hubs and governance.

Day 29: Azure mock and architecture explanation.

Day 30: Full revision: explain one end-to-end data platform.

Readiness test: You are making progress when you can solve a new problem, explain your assumptions, identify failure cases and justify the design—not merely repeat a memorized definition.

SQL Deep-Dive 01 — Relational algebra and query logic

A SELECT query can be understood as a sequence of relational operations: choose a source, filter rows, combine relations, group, project columns and order the result. This mental model helps predict results before execution.

Worked example / reference

```
SELECT department_id, COUNT(*) AS n
FROM Employee
WHERE status = 'Active'
GROUP BY department_id
HAVING COUNT(*) > 2
ORDER BY n DESC;
```

Check your understanding

- Why does WHERE run before grouping?
- What is the difference between projection and selection?
- Why is ORDER BY required for guaranteed output order?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 02 — Keys and referential design

A primary key is a chosen candidate key. Foreign keys enforce valid references. Unique constraints enforce alternate candidate keys. A surrogate key should not replace a meaningful business uniqueness rule.

Worked example / reference

```
CREATE TABLE Department (
  DepartmentID INT PRIMARY KEY,
  DepartmentCode VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE Staff (
  StaffID INT PRIMARY KEY,
  DepartmentID INT NULL,
  FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Check your understanding

- Can a foreign key be nullable?
- When would a composite key be appropriate?
- Why can a surrogate key and unique business key coexist?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 03 — NULL and three-valued logic

SQL predicates can evaluate TRUE, FALSE or UNKNOWN. WHERE keeps only TRUE. This explains why comparisons with NULL do not match and why NOT IN can surprise when a subquery contains NULL.

Worked example / reference

```
SELECT * FROM Employee WHERE ManagerID IS NULL;
SELECT * FROM Employee WHERE COALESCE(Email, '') = '';
```

Check your understanding

- What does UNKNOWN mean?
- Why does NULL = NULL not evaluate to TRUE?
- When is COALESCE appropriate?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 04 — Join strategies and cardinality

Logical join types describe which rows are preserved. Physical plans may use nested loops, hash joins or merge joins. The optimizer chooses based on estimates, indexes, sorting and data sizes.

Worked example / reference

```
SELECT o.OrderID, c.CustomerName
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID;
```

Check your understanding

- What makes a nested-loop join attractive?
- What can cause a hash join to spill?
- How do you detect an unintended many-to-many join?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 05 — Set operations and duplicate semantics

UNION removes duplicate rows; UNION ALL preserves them. INTERSECT returns common rows and EXCEPT returns rows in the first set but not the second where supported. Column counts and compatible types must align.

Worked example / reference

```
SELECT CustomerID FROM OnlineOrders
UNION
SELECT CustomerID FROM StoreOrders;
```

Check your understanding

- Why is UNION ALL often faster?
- When is deduplication a business requirement?
- What must match between set-operation branches?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 06 — Window frames and analytic SQL

Window functions support ranking, offsets and running aggregates. PARTITION BY defines groups, ORDER BY defines sequence and frame clauses define which neighboring rows contribute.

Worked example / reference

```
SELECT OrderDate, Amount,
 SUM(Amount) OVER (
  ORDER BY OrderDate
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
 ) AS RunningAmount
FROM DailySales;
```

Check your understanding

- Why add ROWS explicitly?
- What does LAG return?
- How can ties affect ranking?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 07 — Transactions and anomalies

Concurrent transactions can experience dirty reads, non-repeatable reads, phantom reads and lost updates depending on isolation and locking. The correct isolation level balances consistency and concurrency.

Worked example / reference

```
BEGIN TRANSACTION;
UPDATE Inventory
SET Quantity = Quantity - 1
WHERE ProductID = 20 AND Quantity >= 1;
-- Validate affected row count before commit.
COMMIT;
```

Check your understanding

- Define dirty read.
- What is a lost update?
- Why should transactions be short?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 08 — Indexes and selectivity

An index helps when it narrows the search or provides useful ordering. Selectivity describes how well a predicate distinguishes rows. Low-selectivity columns may still be useful in composite indexes or filtered access patterns.

Worked example / reference

```
CREATE INDEX IX_Order_CustomerDate
ON Orders(CustomerID, OrderDate);
```

Check your understanding

- What is selectivity?
- Why can an index slow inserts?
- What does a covering index achieve?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 09 — Statistics and cardinality estimates

Optimizers use statistics to estimate row counts and choose plans. Stale or unrepresentative statistics can lead to poor join order, memory grants and access paths.

Worked example / reference

```
EXPLAIN SELECT * FROM Orders WHERE CustomerID = 100;
-- SQL Server: inspect the actual execution plan in SSMS.
```

Check your understanding

- What is cardinality estimation?
- Why can skew mislead an optimizer?
- What should be measured after an index change?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 10 — Safe dynamic SQL

Dynamic SQL is sometimes needed for dynamic object names or optional query structure. Parameterize values and allow-list identifiers. Never concatenate untrusted user input into executable SQL.

Worked example / reference

```
-- Conceptual: parameterized values
SELECT * FROM Employee WHERE Email = ?;
```

Check your understanding

- Why cannot table names usually be bound as parameters?
- What is an allow-list?
- What is SQL injection?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 11 — MySQL engine and SQL mode

MySQL behavior depends on server version, SQL mode, character set, collation and storage engine. InnoDB supports transactions and foreign keys. ONLY_FULL_GROUP_BY helps prevent ambiguous aggregate queries.

Worked example / reference

```
SELECT @@version;
SELECT @@sql_mode;
SHOW TABLE STATUS;
```

Check your understanding

- Why prefer InnoDB for transactional tables?
- What is a collation?
- How can SQL mode affect GROUP BY?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 12 — SQL Server T-SQL and batches

T-SQL adds variables, procedural control, TRY/CATCH, temporary tables and system catalog views. GO is a client batch separator, not a server-side SQL statement.

Worked example / reference

```
DECLARE @DepartmentID INT = 3;
SELECT EmployeeID, FullName
FROM dbo.Employee
WHERE DepartmentID = @DepartmentID;
```

Check your understanding

- What is a local variable?
- What does THROW do?
- Why is GO not a T-SQL statement?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 13 — Normalization and anomalies

Normalization reduces redundancy and update anomalies. Functional dependencies describe how one attribute determines another. A practical design balances normalization with workload requirements.

Worked example / reference

```
Employee(EmployeeID, DepartmentID, DepartmentName)
-- DepartmentName depends on DepartmentID, not directly on EmployeeID.
-- Move DepartmentName to Department.
```

Check your understanding

- Describe an update anomaly.
- What is a transitive dependency?
- Why might a warehouse denormalize?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 14 — Data warehousing and grain

A fact table records events at a declared grain. Dimensions provide descriptive context. Measures can be additive, semi-additive or non-additive.

Worked example / reference

```
SELECT d.MonthName, p.Category,
 SUM(f.NetSales) AS NetSales
FROM FactSales f
JOIN DimDate d ON d.DateKey=f.DateKey
JOIN DimProduct p ON p.ProductKey=f.ProductKey
GROUP BY d.MonthName, p.Category;
```

Check your understanding

- Define fact grain.
- Give an example of a semi-additive measure.
- Why use surrogate dimension keys?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 15 — Slowly changing dimensions

SCD Type 1 overwrites attributes; Type 2 creates a new version with effective dates and current-row indicator. The choice depends on whether historical reporting must reflect past attributes.

Worked example / reference

```
CustomerKey | CustomerID | City | StartDate | EndDate | IsCurrent
101         | 55         | Pune | 2024-01-01 | 2025-04-30 | 0
145         | 55         | Bengaluru | 2025-05-01 | NULL | 1
```

Check your understanding

- When is Type 1 suitable?
- What columns support Type 2?
- How does a fact row find the correct historical dimension version?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 16 — Data quality profiling

Profile source data before loading: row counts, null rates, distinct counts, ranges, invalid formats, duplicate keys and referential integrity. Profiling provides evidence for validation rules.

Worked example / reference

```
SELECT COUNT(*) AS rows,
 COUNT(DISTINCT AssetTag) AS distinct_tags,
 SUM(CASE WHEN AssetTag IS NULL THEN 1 ELSE 0 END) AS missing_tags
FROM StagingAsset;
```

Check your understanding

- What does a duplicate-key count reveal?
- Why should invalid rows be quarantined?
- How can distribution drift be detected?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 17 — Query tuning workflow

Tuning starts with a baseline. Capture query text, parameters, data volume, elapsed time, CPU, reads and execution plan. Make one change and re-test correctness and performance.

Worked example / reference

```
1. Reproduce the issue
2. Capture actual plan and metrics
3. Find expensive operator
4. Form one hypothesis
5. Change one thing
6. Compare results
```

Check your understanding

- Why is elapsed time alone insufficient?
- What is parameter sensitivity?
- Why test with realistic data volume?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 18 — Backup, restore and recovery

Backups protect against logical errors and infrastructure failures. Recovery plans specify RPO and RTO. A restore test verifies that backups are usable and procedures are understood.

Worked example / reference

```
Recovery plan fields:
- backup frequency
- retention period
- recovery point objective
- recovery time objective
- restore owner
- test evidence
```

Check your understanding

- Define RPO.
- Define RTO.
- Why is an untested backup risky?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 19 — SQL security and auditing

Database security includes authentication, authorization, encryption, auditing and data minimization. Separate application identities from administrators and grant the least privilege required.

Worked example / reference

```
GRANT SELECT ON dbo.ReportingView TO analyst_role;
-- Avoid granting broad db_owner privileges for read-only reporting.
```

Check your understanding

- What is least privilege?
- Why use a view for reporting access?
- What events should be audited?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 20 — SQL capstone review

A complete SQL solution includes a sound schema, clear grain, constraints, parameterized access, reliable transactions, efficient queries and a recovery plan.

Worked example / reference

```
Checklist:
[ ] primary and foreign keys
[ ] unique business constraints
[ ] NULL behavior documented
[ ] query plan reviewed
[ ] transactions tested
[ ] permissions least-privilege
```

Check your understanding

- Explain your schema to a reviewer.
- Show one report and its expected grain.
- Describe a failure and recovery plan.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 01 — Python object model

Names bind to objects. Mutable objects can change in place; immutable objects cannot. Assignment does not automatically create a copy. Aliasing occurs when two names refer to the same object.

Worked example / reference

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3]
```

Check your understanding

- What is aliasing?
- How does a shallow copy differ from assignment?
- Name three immutable built-in types.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 02 — Numeric precision

Python float uses binary floating point and cannot exactly represent many decimal fractions. Use Decimal for currency and define rounding rules explicitly.

Worked example / reference

```
from decimal import Decimal, ROUND_HALF_UP
price = Decimal('19.99')
qty = Decimal('3')
total = (price * qty).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
```

Check your understanding

- Why use strings to construct Decimal values?
- What is rounding mode?
- When is float appropriate?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 03 — Collection complexity

Lists provide indexed access but membership checks are linear. Sets and dictionaries offer average constant-time membership and key lookup. Algorithm choices matter for large data.

Worked example / reference

```
seen = set()
for record in records:
    key = record['id']
    if key not in seen:
        seen.add(key)
```

Check your understanding

- What is O(n)?
- Why is set membership useful?
- When can a set not be used as a key?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 04 — Function contracts and type hints

A function contract describes inputs, outputs, side effects and errors. Type hints improve readability and tooling but are not runtime enforcement by default.

Worked example / reference

```
def calculate_tax(amount: float, rate: float) -> float:
    """Return tax for a non-negative amount."""
    if amount < 0:
        raise ValueError('amount must be non-negative')
    return amount * rate
```

Check your understanding

- What is a side effect?
- Why document exceptions?
- What does a type checker do?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 05 — Iterators and lazy evaluation

An iterator produces values on demand. Lazy processing can reduce memory use, but it means errors may occur later when the iterator is consumed.

Worked example / reference

```
def batches(items, size):
    batch = []
    for item in items:
        batch.append(item)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
```

Check your understanding

- Why is lazy evaluation memory efficient?
- What happens after generator exhaustion?
- How can batch size affect throughput?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 06 — Exceptions and error boundaries

Catch exceptions where you can recover, add context or translate them into a domain-specific error. Avoid catching everything and continuing with corrupted state.

Worked example / reference

```
try:
    rows = read_source(path)
except FileNotFoundError as exc:
    logger.error('Source missing: %s', path)
    raise PipelineInputError(path) from exc
```

Check your understanding

- What is exception chaining?
- Why not suppress all errors?
- When should a pipeline fail fast?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 07 — File encoding and robust ingestion

Text data can have different encodings, delimiters, line endings and quoting. Specify encoding, validate headers and test representative source files.

Worked example / reference

```
with open('input.csv', newline='', encoding='utf-8-sig') as f:
    reader = csv.DictReader(f)
    required = {'asset_tag', 'category'}
    if not required.issubset(reader.fieldnames or []):
        raise ValueError('Required headers missing')
```

Check your understanding

- Why might utf-8-sig be useful?
- How do quoted commas affect CSV parsing?
- What should happen when a required column is absent?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 08 — JSON schema and API payloads

JSON is flexible but does not enforce business schema by itself. Validate required keys, types, ranges and schema version before transformation.

Worked example / reference

```
payload = {'event_id': 'e1', 'version': 2, 'amount': 15.5}
required = {'event_id', 'version', 'amount'}
missing = required - payload.keys()
if missing:
    raise ValueError(f'Missing keys: {missing}')
```

Check your understanding

- Why include schema version?
- How do you handle unknown fields?
- What is backward-compatible schema evolution?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 09 — Pandas indexing and selection

Pandas label selection uses loc; positional selection uses iloc. Boolean masks filter rows. Chained assignment can create ambiguity; prefer a single .loc assignment.

Worked example / reference

```
mask = df['salary'].ge(50000)
df.loc[mask, 'salary_band'] = 'High'
df.loc[~mask, 'salary_band'] = 'Other'
```

Check your understanding

- What is the difference between loc and iloc?
- Why avoid chained assignment?
- How do you select rows with missing values?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 10 — Pandas merge validation

DataFrame merge can create unexpected row multiplication if keys are not unique. Use validate to assert expected relationship and indicator to inspect unmatched records.

Worked example / reference

```
result = left.merge(
    right, on='department_id', how='left',
    validate='many_to_one', indicator=True
)
print(result['_merge'].value_counts())
```

Check your understanding

- What does many_to_one mean?
- How do you find unmatched left rows?
- Why check duplicate keys before merging?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 11 — Pandas missing data

Missing values may mean unknown, not applicable, not collected or failed conversion. Do not automatically fill every missing value with zero.

Worked example / reference

```
df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
invalid = df[df['amount'].isna()]
clean = df.dropna(subset=['record_id'])
```

Check your understanding

- Why can zero be a misleading replacement?
- How do you flag parse failures?
- When is dropna justified?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 12 — Pandas performance

Vectorized operations are usually faster than Python-level row loops. Use categorical dtypes for repeated labels when appropriate, select needed columns and process large files in chunks.

Worked example / reference

```
for chunk in pd.read_csv('large.csv', chunksize=100_000):
    clean = chunk.loc[chunk['status'].eq('Valid')]
    write_chunk(clean)
```

Check your understanding

- What is vectorization?
- When can categorical dtype save memory?
- What is a chunked read?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 13 — Testing strategy

Unit tests verify functions; integration tests verify connected components; end-to-end tests verify complete workflows. Include boundary values, malformed inputs and repeat-run behavior.

Worked example / reference

```
def test_empty_tag_rejected():
    with pytest.raises(ValueError):
        normalize_tag('   ')
```

Check your understanding

- What is a fixture?
- What is a regression test?
- How do you test a database integration?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 14 — Logging and observability

Logs describe events; metrics quantify behavior; traces connect operations across systems. A run ID links pipeline logs and records. Avoid logging credentials or sensitive payloads.

Worked example / reference

```
logger.info('load_complete', extra={
    'run_id': run_id,
    'source_count': source_count,
    'target_count': target_count
})
```

Check your understanding

- What is structured logging?
- What is a metric?
- Why include a run ID?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 15 — API reliability

APIs can fail due to timeouts, throttling, transient server errors and schema changes. Use timeouts, bounded retries, backoff and pagination.

Worked example / reference

```
for attempt in range(4):
    try:
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        break
    except requests.Timeout:
        if attempt == 3:
            raise
        time.sleep(2 ** attempt)
```

Check your understanding

- Why not retry every 4xx response?
- What is exponential backoff?
- How do you avoid duplicate POST operations?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 16 — Database transactions from Python

Database code should use parameterized statements, commit only after success and roll back on failure. Connection pooling can improve performance in long-running applications.

Worked example / reference

```
try:
    cursor.execute('UPDATE Asset SET Status = ? WHERE AssetID = ?', (status, asset_id))
    connection.commit()
except Exception:
    connection.rollback()
    raise
```

Check your understanding

- Why are placeholders driver-specific?
- When should a transaction begin?
- What is connection pooling?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 17 — Concurrency and parallelism

Threads, processes and async IO solve different problems. IO-bound workloads may benefit from async or threads; CPU-bound Python workloads may need processes or distributed compute.

Worked example / reference

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=4) as pool:
    results = list(pool.map(fetch_one, urls))
```

Check your understanding

- What is IO-bound work?
- What is CPU-bound work?
- Why cap worker count?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 18 — Pipeline idempotency

A retry should not create duplicate output. Use stable keys, deterministic transformations, merge/upsert semantics, checkpoints and run metadata.

Worked example / reference

```
def merge_by_key(existing, incoming):
    result = dict(existing)
    for row in incoming:
        result[row['id']] = row
    return result
```

Check your understanding

- What is a natural idempotency key?
- What happens if the checkpoint is saved too early?
- How do you handle deletes?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 19 — Python capstone review

A robust Python data pipeline is modular, typed where helpful, tested, logged, secure, restartable and memory-aware.

Worked example / reference

```
Extract -> Validate -> Transform -> Stage -> Publish -> Reconcile -> Checkpoint
```

Check your understanding

- Describe one source failure.
- Explain your retry strategy.
- Show how rejected records are reported.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 01 — Azure resource hierarchy

Management groups organize subscriptions; subscriptions provide billing and management boundaries; resource groups organize resources; resources are service instances. Naming and tags help operations.

Worked example / reference

```
Tenant
  Management Group
    Subscription
      Resource Group
        Storage / Data Factory / Key Vault
```

Check your understanding

- Where can Azure Policy be assigned?
- What is a subscription boundary?
- Why use environment tags?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 02 — Identity and managed identities

Managed identities provide an Azure-managed identity for supported workloads. They avoid embedding credentials in code and can be granted scoped roles.

Worked example / reference

```
Data Factory managed identity
  -> Storage data role on approved filesystem
  -> Key Vault access only if needed
```

Check your understanding

- System-assigned versus user-assigned identity?
- Why is Owner usually excessive?
- What is a service principal?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 03 — RBAC and data-plane permissions

Azure RBAC controls access at resource scopes; services can also have data-plane authorization such as ADLS ACLs. Network reachability and authorization are separate controls.

Worked example / reference

```
Control plane: create/configure a storage account
Data plane: read/write blobs and directories
```

Check your understanding

- Can a user create a resource but lack data read access?
- What is an ACL?
- Why scope roles narrowly?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 04 — Storage redundancy and recovery

LRS, ZRS, GRS and GZRS provide different replication patterns. Redundancy does not replace backup, restore testing or application-level recovery planning.

Worked example / reference

```
Select based on:
- regional availability
- data residency
- RPO/RTO
- cost
- supported service features
```

Check your understanding

- What is zone redundancy?
- What does geo-replication protect against?
- Why test restore?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 05 — ADLS Gen2 hierarchy and ACLs

Hierarchical namespace enables directory/file semantics and ACLs. Directory execute permissions are needed to traverse paths. Design permissions and inherited ACLs carefully.

Worked example / reference

```
/filesystem/bronze/source/entity/ingest_date=YYYY-MM-DD/
```

Check your understanding

- What is hierarchical namespace?
- Why can ACL inheritance matter?
- How should raw data access differ from curated data access?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 06 — File formats and partitioning

CSV is simple but expensive to parse and weakly typed. Parquet is columnar and efficient for analytics. Partition by commonly filtered low-to-moderate cardinality columns.

Worked example / reference

```
Example layout:
/orders/year=2026/month=10/day=31/part-*.parquet
```

Check your understanding

- Why not partition by every customer ID?
- What is column pruning?
- Why avoid tiny files?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 07 — ADF orchestration model

ADF uses pipelines, activities, linked services, datasets, triggers and integration runtimes. Parameters allow reuse; monitoring and activity outputs support diagnosis.

Worked example / reference

```
Pipeline -> Copy -> Validate -> Transform -> Publish -> Notify
```

Check your understanding

- What is a linked service?
- When is self-hosted IR needed?
- What is the difference between trigger and activity?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 08 — ADF watermark architecture

A robust incremental load reads the previous watermark, extracts changes, stages, validates, merges, then updates control state only after successful publication.

Worked example / reference

```
Read watermark
Extract delta
Stage and validate
Merge target
Reconcile counts
Commit watermark
```

Check your understanding

- What if two records share a timestamp?
- How do you handle deletes?
- Why stage before merge?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 09 — ADF parameters and expressions

Parameters are immutable run inputs; variables can change during execution. Dynamic expressions create reusable paths and query values. Test time zones and boundary dates.

Worked example / reference

```
@concat('landing/orders/year=', formatDateTime(utcNow(),'yyyy'), '/month=', formatDateTime(utcNow(),'MM'))
```

Check your understanding

- Why parameterize folder names?
- What is a trigger time zone concern?
- How do you test an expression?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 10 — ADF integration runtime selection

Azure IR handles cloud data movement and dispatch; self-hosted IR reaches private/on-premises networks; Azure-SSIS IR supports SSIS packages. Connectivity and data locality drive selection.

Worked example / reference

```
On-prem SQL Server -> Self-hosted IR -> Azure Data Lake
```

Check your understanding

- Where does self-hosted IR run?
- What operational responsibility does it add?
- How can network firewalls affect it?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 11 — Databricks Spark execution

Spark applications use a driver and executors. Transformations are lazy, actions trigger execution, and shuffles redistribute data across partitions.

Worked example / reference

```
df.filter('status = "OK"').groupBy('region').count().show()
```

Check your understanding

- What is a Spark action?
- What is a shuffle?
- Why avoid collect on large data?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 12 — Spark optimization

Inspect plans, stages, skew, spill, partition sizes and file counts. Use broadcast joins for small dimensions where safe; avoid caching data used only once.

Worked example / reference

```
df.explain('formatted')
# Inspect Spark UI for stages, shuffle and task duration
```

Check your understanding

- What is skew?
- When is broadcast join useful?
- Why can too many partitions hurt?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 13 — Delta Lake table reliability

Delta Lake adds a transaction log and table-level capabilities such as ACID operations and schema enforcement. MERGE supports common upsert patterns in supported environments.

Worked example / reference

```
MERGE INTO target t
USING updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Check your understanding

- Why use MERGE?
- What is schema enforcement?
- What is time travel useful for?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 01 — Relational algebra and query logic

A SELECT query can be understood as a sequence of relational operations: choose a source, filter rows, combine relations, group, project columns and order the result. This mental model helps predict results before execution.

Worked example / reference

```
SELECT department_id, COUNT(*) AS n
FROM Employee
WHERE status = 'Active'
GROUP BY department_id
HAVING COUNT(*) > 2
ORDER BY n DESC;
```

Check your understanding

- Why does WHERE run before grouping?
- What is the difference between projection and selection?
- Why is ORDER BY required for guaranteed output order?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 02 — Keys and referential design

A primary key is a chosen candidate key. Foreign keys enforce valid references. Unique constraints enforce alternate candidate keys. A surrogate key should not replace a meaningful business uniqueness rule.

Worked example / reference

```
CREATE TABLE Department (
  DepartmentID INT PRIMARY KEY,
  DepartmentCode VARCHAR(20) NOT NULL UNIQUE
);
CREATE TABLE Staff (
  StaffID INT PRIMARY KEY,
  DepartmentID INT NULL,
  FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);
```

Check your understanding

- Can a foreign key be nullable?
- When would a composite key be appropriate?
- Why can a surrogate key and unique business key coexist?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 03 — NULL and three-valued logic

SQL predicates can evaluate TRUE, FALSE or UNKNOWN. WHERE keeps only TRUE. This explains why comparisons with NULL do not match and why NOT IN can surprise when a subquery contains NULL.

Worked example / reference

```
SELECT * FROM Employee WHERE ManagerID IS NULL;
SELECT * FROM Employee WHERE COALESCE(Email, '') = '';
```

Check your understanding

- What does UNKNOWN mean?
- Why does NULL = NULL not evaluate to TRUE?
- When is COALESCE appropriate?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 04 — Join strategies and cardinality

Logical join types describe which rows are preserved. Physical plans may use nested loops, hash joins or merge joins. The optimizer chooses based on estimates, indexes, sorting and data sizes.

Worked example / reference

```
SELECT o.OrderID, c.CustomerName
FROM Orders o
JOIN Customers c ON c.CustomerID = o.CustomerID;
```

Check your understanding

- What makes a nested-loop join attractive?
- What can cause a hash join to spill?
- How do you detect an unintended many-to-many join?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 05 — Set operations and duplicate semantics

UNION removes duplicate rows; UNION ALL preserves them. INTERSECT returns common rows and EXCEPT returns rows in the first set but not the second where supported. Column counts and compatible types must align.

Worked example / reference

```
SELECT CustomerID FROM OnlineOrders
UNION
SELECT CustomerID FROM StoreOrders;
```

Check your understanding

- Why is UNION ALL often faster?
- When is deduplication a business requirement?
- What must match between set-operation branches?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 06 — Window frames and analytic SQL

Window functions support ranking, offsets and running aggregates. PARTITION BY defines groups, ORDER BY defines sequence and frame clauses define which neighboring rows contribute.

Worked example / reference

```
SELECT OrderDate, Amount,
 SUM(Amount) OVER (
  ORDER BY OrderDate
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
 ) AS RunningAmount
FROM DailySales;
```

Check your understanding

- Why add ROWS explicitly?
- What does LAG return?
- How can ties affect ranking?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 07 — Transactions and anomalies

Concurrent transactions can experience dirty reads, non-repeatable reads, phantom reads and lost updates depending on isolation and locking. The correct isolation level balances consistency and concurrency.

Worked example / reference

```
BEGIN TRANSACTION;
UPDATE Inventory
SET Quantity = Quantity - 1
WHERE ProductID = 20 AND Quantity >= 1;
-- Validate affected row count before commit.
COMMIT;
```

Check your understanding

- Define dirty read.
- What is a lost update?
- Why should transactions be short?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 08 — Indexes and selectivity

An index helps when it narrows the search or provides useful ordering. Selectivity describes how well a predicate distinguishes rows. Low-selectivity columns may still be useful in composite indexes or filtered access patterns.

Worked example / reference

```
CREATE INDEX IX_Order_CustomerDate
ON Orders(CustomerID, OrderDate);
```

Check your understanding

- What is selectivity?
- Why can an index slow inserts?
- What does a covering index achieve?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 09 — Statistics and cardinality estimates

Optimizers use statistics to estimate row counts and choose plans. Stale or unrepresentative statistics can lead to poor join order, memory grants and access paths.

Worked example / reference

```
EXPLAIN SELECT * FROM Orders WHERE CustomerID = 100;
-- SQL Server: inspect the actual execution plan in SSMS.
```

Check your understanding

- What is cardinality estimation?
- Why can skew mislead an optimizer?
- What should be measured after an index change?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 10 — Safe dynamic SQL

Dynamic SQL is sometimes needed for dynamic object names or optional query structure. Parameterize values and allow-list identifiers. Never concatenate untrusted user input into executable SQL.

Worked example / reference

```
-- Conceptual: parameterized values
SELECT * FROM Employee WHERE Email = ?;
```

Check your understanding

- Why cannot table names usually be bound as parameters?
- What is an allow-list?
- What is SQL injection?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 11 — MySQL engine and SQL mode

MySQL behavior depends on server version, SQL mode, character set, collation and storage engine. InnoDB supports transactions and foreign keys. ONLY_FULL_GROUP_BY helps prevent ambiguous aggregate queries.

Worked example / reference

```
SELECT @@version;
SELECT @@sql_mode;
SHOW TABLE STATUS;
```

Check your understanding

- Why prefer InnoDB for transactional tables?
- What is a collation?
- How can SQL mode affect GROUP BY?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 12 — SQL Server T-SQL and batches

T-SQL adds variables, procedural control, TRY/CATCH, temporary tables and system catalog views. GO is a client batch separator, not a server-side SQL statement.

Worked example / reference

```
DECLARE @DepartmentID INT = 3;
SELECT EmployeeID, FullName
FROM dbo.Employee
WHERE DepartmentID = @DepartmentID;
```

Check your understanding

- What is a local variable?
- What does THROW do?
- Why is GO not a T-SQL statement?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 13 — Normalization and anomalies

Normalization reduces redundancy and update anomalies. Functional dependencies describe how one attribute determines another. A practical design balances normalization with workload requirements.

Worked example / reference

```
Employee(EmployeeID, DepartmentID, DepartmentName)
-- DepartmentName depends on DepartmentID, not directly on EmployeeID.
-- Move DepartmentName to Department.
```

Check your understanding

- Describe an update anomaly.
- What is a transitive dependency?
- Why might a warehouse denormalize?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 14 — Data warehousing and grain

A fact table records events at a declared grain. Dimensions provide descriptive context. Measures can be additive, semi-additive or non-additive.

Worked example / reference

```
SELECT d.MonthName, p.Category,
 SUM(f.NetSales) AS NetSales
FROM FactSales f
JOIN DimDate d ON d.DateKey=f.DateKey
JOIN DimProduct p ON p.ProductKey=f.ProductKey
GROUP BY d.MonthName, p.Category;
```

Check your understanding

- Define fact grain.
- Give an example of a semi-additive measure.
- Why use surrogate dimension keys?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 15 — Slowly changing dimensions

SCD Type 1 overwrites attributes; Type 2 creates a new version with effective dates and current-row indicator. The choice depends on whether historical reporting must reflect past attributes.

Worked example / reference

```
CustomerKey | CustomerID | City | StartDate | EndDate | IsCurrent
101         | 55         | Pune | 2024-01-01 | 2025-04-30 | 0
145         | 55         | Bengaluru | 2025-05-01 | NULL | 1
```

Check your understanding

- When is Type 1 suitable?
- What columns support Type 2?
- How does a fact row find the correct historical dimension version?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 16 — Data quality profiling

Profile source data before loading: row counts, null rates, distinct counts, ranges, invalid formats, duplicate keys and referential integrity. Profiling provides evidence for validation rules.

Worked example / reference

```
SELECT COUNT(*) AS rows,
 COUNT(DISTINCT AssetTag) AS distinct_tags,
 SUM(CASE WHEN AssetTag IS NULL THEN 1 ELSE 0 END) AS missing_tags
FROM StagingAsset;
```

Check your understanding

- What does a duplicate-key count reveal?
- Why should invalid rows be quarantined?
- How can distribution drift be detected?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 17 — Query tuning workflow

Tuning starts with a baseline. Capture query text, parameters, data volume, elapsed time, CPU, reads and execution plan. Make one change and re-test correctness and performance.

Worked example / reference

```
1. Reproduce the issue
2. Capture actual plan and metrics
3. Find expensive operator
4. Form one hypothesis
5. Change one thing
6. Compare results
```

Check your understanding

- Why is elapsed time alone insufficient?
- What is parameter sensitivity?
- Why test with realistic data volume?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 18 — Backup, restore and recovery

Backups protect against logical errors and infrastructure failures. Recovery plans specify RPO and RTO. A restore test verifies that backups are usable and procedures are understood.

Worked example / reference

```
Recovery plan fields:
- backup frequency
- retention period
- recovery point objective
- recovery time objective
- restore owner
- test evidence
```

Check your understanding

- Define RPO.
- Define RTO.
- Why is an untested backup risky?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 19 — SQL security and auditing

Database security includes authentication, authorization, encryption, auditing and data minimization. Separate application identities from administrators and grant the least privilege required.

Worked example / reference

```
GRANT SELECT ON dbo.ReportingView TO analyst_role;
-- Avoid granting broad db_owner privileges for read-only reporting.
```

Check your understanding

- What is least privilege?
- Why use a view for reporting access?
- What events should be audited?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

SQL Deep-Dive 20 — SQL capstone review

A complete SQL solution includes a sound schema, clear grain, constraints, parameterized access, reliable transactions, efficient queries and a recovery plan.

Worked example / reference

```
Checklist:
[ ] primary and foreign keys
[ ] unique business constraints
[ ] NULL behavior documented
[ ] query plan reviewed
[ ] transactions tested
[ ] permissions least-privilege
```

Check your understanding

- Explain your schema to a reviewer.
- Show one report and its expected grain.
- Describe a failure and recovery plan.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 01 — Python object model

Names bind to objects. Mutable objects can change in place; immutable objects cannot. Assignment does not automatically create a copy. Aliasing occurs when two names refer to the same object.

Worked example / reference

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3]
```

Check your understanding

- What is aliasing?
- How does a shallow copy differ from assignment?
- Name three immutable built-in types.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 02 — Numeric precision

Python float uses binary floating point and cannot exactly represent many decimal fractions. Use Decimal for currency and define rounding rules explicitly.

Worked example / reference

```
from decimal import Decimal, ROUND_HALF_UP
price = Decimal('19.99')
qty = Decimal('3')
total = (price * qty).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)
```

Check your understanding

- Why use strings to construct Decimal values?
- What is rounding mode?
- When is float appropriate?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 03 — Collection complexity

Lists provide indexed access but membership checks are linear. Sets and dictionaries offer average constant-time membership and key lookup. Algorithm choices matter for large data.

Worked example / reference

```
seen = set()
for record in records:
    key = record['id']
    if key not in seen:
        seen.add(key)
```

Check your understanding

- What is O(n)?
- Why is set membership useful?
- When can a set not be used as a key?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 04 — Function contracts and type hints

A function contract describes inputs, outputs, side effects and errors. Type hints improve readability and tooling but are not runtime enforcement by default.

Worked example / reference

```
def calculate_tax(amount: float, rate: float) -> float:
    """Return tax for a non-negative amount."""
    if amount < 0:
        raise ValueError('amount must be non-negative')
    return amount * rate
```

Check your understanding

- What is a side effect?
- Why document exceptions?
- What does a type checker do?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 05 — Iterators and lazy evaluation

An iterator produces values on demand. Lazy processing can reduce memory use, but it means errors may occur later when the iterator is consumed.

Worked example / reference

```
def batches(items, size):
    batch = []
    for item in items:
        batch.append(item)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
```

Check your understanding

- Why is lazy evaluation memory efficient?
- What happens after generator exhaustion?
- How can batch size affect throughput?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 06 — Exceptions and error boundaries

Catch exceptions where you can recover, add context or translate them into a domain-specific error. Avoid catching everything and continuing with corrupted state.

Worked example / reference

```
try:
    rows = read_source(path)
except FileNotFoundError as exc:
    logger.error('Source missing: %s', path)
    raise PipelineInputError(path) from exc
```

Check your understanding

- What is exception chaining?
- Why not suppress all errors?
- When should a pipeline fail fast?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 07 — File encoding and robust ingestion

Text data can have different encodings, delimiters, line endings and quoting. Specify encoding, validate headers and test representative source files.

Worked example / reference

```
with open('input.csv', newline='', encoding='utf-8-sig') as f:
    reader = csv.DictReader(f)
    required = {'asset_tag', 'category'}
    if not required.issubset(reader.fieldnames or []):
        raise ValueError('Required headers missing')
```

Check your understanding

- Why might utf-8-sig be useful?
- How do quoted commas affect CSV parsing?
- What should happen when a required column is absent?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 08 — JSON schema and API payloads

JSON is flexible but does not enforce business schema by itself. Validate required keys, types, ranges and schema version before transformation.

Worked example / reference

```
payload = {'event_id': 'e1', 'version': 2, 'amount': 15.5}
required = {'event_id', 'version', 'amount'}
missing = required - payload.keys()
if missing:
    raise ValueError(f'Missing keys: {missing}')
```

Check your understanding

- Why include schema version?
- How do you handle unknown fields?
- What is backward-compatible schema evolution?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 09 — Pandas indexing and selection

Pandas label selection uses loc; positional selection uses iloc. Boolean masks filter rows. Chained assignment can create ambiguity; prefer a single .loc assignment.

Worked example / reference

```
mask = df['salary'].ge(50000)
df.loc[mask, 'salary_band'] = 'High'
df.loc[~mask, 'salary_band'] = 'Other'
```

Check your understanding

- What is the difference between loc and iloc?
- Why avoid chained assignment?
- How do you select rows with missing values?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 10 — Pandas merge validation

DataFrame merge can create unexpected row multiplication if keys are not unique. Use validate to assert expected relationship and indicator to inspect unmatched records.

Worked example / reference

```
result = left.merge(
    right, on='department_id', how='left',
    validate='many_to_one', indicator=True
)
print(result['_merge'].value_counts())
```

Check your understanding

- What does many_to_one mean?
- How do you find unmatched left rows?
- Why check duplicate keys before merging?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 11 — Pandas missing data

Missing values may mean unknown, not applicable, not collected or failed conversion. Do not automatically fill every missing value with zero.

Worked example / reference

```
df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
invalid = df[df['amount'].isna()]
clean = df.dropna(subset=['record_id'])
```

Check your understanding

- Why can zero be a misleading replacement?
- How do you flag parse failures?
- When is dropna justified?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 12 — Pandas performance

Vectorized operations are usually faster than Python-level row loops. Use categorical dtypes for repeated labels when appropriate, select needed columns and process large files in chunks.

Worked example / reference

```
for chunk in pd.read_csv('large.csv', chunksize=100_000):
    clean = chunk.loc[chunk['status'].eq('Valid')]
    write_chunk(clean)
```

Check your understanding

- What is vectorization?
- When can categorical dtype save memory?
- What is a chunked read?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 13 — Testing strategy

Unit tests verify functions; integration tests verify connected components; end-to-end tests verify complete workflows. Include boundary values, malformed inputs and repeat-run behavior.

Worked example / reference

```
def test_empty_tag_rejected():
    with pytest.raises(ValueError):
        normalize_tag('   ')
```

Check your understanding

- What is a fixture?
- What is a regression test?
- How do you test a database integration?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 14 — Logging and observability

Logs describe events; metrics quantify behavior; traces connect operations across systems. A run ID links pipeline logs and records. Avoid logging credentials or sensitive payloads.

Worked example / reference

```
logger.info('load_complete', extra={
    'run_id': run_id,
    'source_count': source_count,
    'target_count': target_count
})
```

Check your understanding

- What is structured logging?
- What is a metric?
- Why include a run ID?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 15 — API reliability

APIs can fail due to timeouts, throttling, transient server errors and schema changes. Use timeouts, bounded retries, backoff and pagination.

Worked example / reference

```
for attempt in range(4):
    try:
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        break
    except requests.Timeout:
        if attempt == 3:
            raise
        time.sleep(2 ** attempt)
```

Check your understanding

- Why not retry every 4xx response?
- What is exponential backoff?
- How do you avoid duplicate POST operations?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 16 — Database transactions from Python

Database code should use parameterized statements, commit only after success and roll back on failure. Connection pooling can improve performance in long-running applications.

Worked example / reference

```
try:
    cursor.execute('UPDATE Asset SET Status = ? WHERE AssetID = ?', (status, asset_id))
    connection.commit()
except Exception:
    connection.rollback()
    raise
```

Check your understanding

- Why are placeholders driver-specific?
- When should a transaction begin?
- What is connection pooling?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 17 — Concurrency and parallelism

Threads, processes and async IO solve different problems. IO-bound workloads may benefit from async or threads; CPU-bound Python workloads may need processes or distributed compute.

Worked example / reference

```
from concurrent.futures import ThreadPoolExecutor
with ThreadPoolExecutor(max_workers=4) as pool:
    results = list(pool.map(fetch_one, urls))
```

Check your understanding

- What is IO-bound work?
- What is CPU-bound work?
- Why cap worker count?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 18 — Pipeline idempotency

A retry should not create duplicate output. Use stable keys, deterministic transformations, merge/upsert semantics, checkpoints and run metadata.

Worked example / reference

```
def merge_by_key(existing, incoming):
    result = dict(existing)
    for row in incoming:
        result[row['id']] = row
    return result
```

Check your understanding

- What is a natural idempotency key?
- What happens if the checkpoint is saved too early?
- How do you handle deletes?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Python Deep-Dive 19 — Python capstone review

A robust Python data pipeline is modular, typed where helpful, tested, logged, secure, restartable and memory-aware.

Worked example / reference

```
Extract -> Validate -> Transform -> Stage -> Publish -> Reconcile -> Checkpoint
```

Check your understanding

- Describe one source failure.
- Explain your retry strategy.
- Show how rejected records are reported.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 01 — Azure resource hierarchy

Management groups organize subscriptions; subscriptions provide billing and management boundaries; resource groups organize resources; resources are service instances. Naming and tags help operations.

Worked example / reference

```
Tenant
  Management Group
    Subscription
      Resource Group
        Storage / Data Factory / Key Vault
```

Check your understanding

- Where can Azure Policy be assigned?
- What is a subscription boundary?
- Why use environment tags?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 02 — Identity and managed identities

Managed identities provide an Azure-managed identity for supported workloads. They avoid embedding credentials in code and can be granted scoped roles.

Worked example / reference

```
Data Factory managed identity
  -> Storage data role on approved filesystem
  -> Key Vault access only if needed
```

Check your understanding

- System-assigned versus user-assigned identity?
- Why is Owner usually excessive?
- What is a service principal?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 03 — RBAC and data-plane permissions

Azure RBAC controls access at resource scopes; services can also have data-plane authorization such as ADLS ACLs. Network reachability and authorization are separate controls.

Worked example / reference

```
Control plane: create/configure a storage account
Data plane: read/write blobs and directories
```

Check your understanding

- Can a user create a resource but lack data read access?
- What is an ACL?
- Why scope roles narrowly?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 04 — Storage redundancy and recovery

LRS, ZRS, GRS and GZRS provide different replication patterns. Redundancy does not replace backup, restore testing or application-level recovery planning.

Worked example / reference

```
Select based on:
- regional availability
- data residency
- RPO/RTO
- cost
- supported service features
```

Check your understanding

- What is zone redundancy?
- What does geo-replication protect against?
- Why test restore?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 05 — ADLS Gen2 hierarchy and ACLs

Hierarchical namespace enables directory/file semantics and ACLs. Directory execute permissions are needed to traverse paths. Design permissions and inherited ACLs carefully.

Worked example / reference

```
/filesystem/bronze/source/entity/ingest_date=YYYY-MM-DD/
```

Check your understanding

- What is hierarchical namespace?
- Why can ACL inheritance matter?
- How should raw data access differ from curated data access?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 06 — File formats and partitioning

CSV is simple but expensive to parse and weakly typed. Parquet is columnar and efficient for analytics. Partition by commonly filtered low-to-moderate cardinality columns.

Worked example / reference

```
Example layout:
/orders/year=2026/month=10/day=31/part-*.parquet
```

Check your understanding

- Why not partition by every customer ID?
- What is column pruning?
- Why avoid tiny files?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 07 — ADF orchestration model

ADF uses pipelines, activities, linked services, datasets, triggers and integration runtimes. Parameters allow reuse; monitoring and activity outputs support diagnosis.

Worked example / reference

```
Pipeline -> Copy -> Validate -> Transform -> Publish -> Notify
```

Check your understanding

- What is a linked service?
- When is self-hosted IR needed?
- What is the difference between trigger and activity?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 08 — ADF watermark architecture

A robust incremental load reads the previous watermark, extracts changes, stages, validates, merges, then updates control state only after successful publication.

Worked example / reference

```
Read watermark
Extract delta
Stage and validate
Merge target
Reconcile counts
Commit watermark
```

Check your understanding

- What if two records share a timestamp?
- How do you handle deletes?
- Why stage before merge?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 09 — ADF parameters and expressions

Parameters are immutable run inputs; variables can change during execution. Dynamic expressions create reusable paths and query values. Test time zones and boundary dates.

Worked example / reference

```
@concat('landing/orders/year=', formatDateTime(utcNow(),'yyyy'), '/month=', formatDateTime(utcNow(),'MM'))
```

Check your understanding

- Why parameterize folder names?
- What is a trigger time zone concern?
- How do you test an expression?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 10 — ADF integration runtime selection

Azure IR handles cloud data movement and dispatch; self-hosted IR reaches private/on-premises networks; Azure-SSIS IR supports SSIS packages. Connectivity and data locality drive selection.

Worked example / reference

```
On-prem SQL Server -> Self-hosted IR -> Azure Data Lake
```

Check your understanding

- Where does self-hosted IR run?
- What operational responsibility does it add?
- How can network firewalls affect it?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 11 — Databricks Spark execution

Spark applications use a driver and executors. Transformations are lazy, actions trigger execution, and shuffles redistribute data across partitions.

Worked example / reference

```
df.filter('status = "OK"').groupBy('region').count().show()
```

Check your understanding

- What is a Spark action?
- What is a shuffle?
- Why avoid collect on large data?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 12 — Spark optimization

Inspect plans, stages, skew, spill, partition sizes and file counts. Use broadcast joins for small dimensions where safe; avoid caching data used only once.

Worked example / reference

```
df.explain('formatted')
# Inspect Spark UI for stages, shuffle and task duration
```

Check your understanding

- What is skew?
- When is broadcast join useful?
- Why can too many partitions hurt?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 13 — Delta Lake table reliability

Delta Lake adds a transaction log and table-level capabilities such as ACID operations and schema enforcement. MERGE supports common upsert patterns in supported environments.

Worked example / reference

```
MERGE INTO target t
USING updates s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Check your understanding

- Why use MERGE?
- What is schema enforcement?
- What is time travel useful for?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 14 — Synapse SQL choices

Serverless SQL queries files without a provisioned dedicated warehouse. Dedicated SQL pool is an MPP warehouse. Spark supports distributed code and data engineering notebooks.

Worked example / reference

```
Concept checklist:
- Define the requirement
- Select the service or technique
- Validate security and failure behavior
```

Check your understanding

- Which service fits ad hoc lake exploration?
- What is MPP?
- Why choose distribution keys carefully?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 15 — Synapse distribution strategies

Hash distribution can colocate rows for joins; round-robin spreads rows for staging; replicated distribution copies small tables to compute nodes. Distribution choices should be tested.

Worked example / reference

```
Large fact table: consider hash distribution on a frequently joined key.
Small dimension: consider replication where suitable.
```

Check your understanding

- What causes data movement?
- When is round-robin useful?
- What is skewed distribution?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 16 — Fabric OneLake and items

Fabric integrates data engineering, integration, warehouse, real-time analytics and BI. OneLake provides a unified logical data lake; workspaces contain Fabric items.

Worked example / reference

```
Workspace
  Lakehouse
  Notebook
  Pipeline
  Warehouse
  Semantic Model
```

Check your understanding

- What is a shortcut?
- What is capacity?
- How does a semantic model help users?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 17 — Lakehouse versus Warehouse

Lakehouse supports files, tables and Spark-oriented engineering; Warehouse provides a relational SQL analytics experience. The best choice depends on users, modeling and workload.

Worked example / reference

```
Concept checklist:
- Define the requirement
- Select the service or technique
- Validate security and failure behavior
```

Check your understanding

- Choose for Spark-first transformations.
- Choose for SQL-centric warehouse modeling.
- How might both coexist?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 18 — Streaming and Event Hubs

Event Hubs ingests high-throughput event streams. Partitions provide parallelism and order within a partition; consumer groups provide independent read views.

Worked example / reference

```
Producer -> Event Hub -> Consumer group -> Stream processor -> Sink
```

Check your understanding

- What is consumer lag?
- How does partition key affect ordering?
- Why deduplicate events?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 19 — Security and private networking

Use Entra identity, managed identity, RBAC, ACLs, Key Vault and private endpoints. Private networking does not replace authorization. Configure private DNS and test service connectivity.

Worked example / reference

```
Concept checklist:
- Define the requirement
- Select the service or technique
- Validate security and failure behavior
```

Check your understanding

- Why use Key Vault?
- What does a private endpoint do?
- Why is public access restriction important?

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.

Azure Deep-Dive 20 — Azure capstone and operations

An enterprise pipeline needs ownership, data contracts, quality rules, lineage, alerts, cost controls and a tested recovery path—not just successful copy activity.

Worked example / reference

```
Source -> Ingest -> Landing -> Validate -> Transform -> Curate -> Serve
       Identity + Network + Governance + Monitoring + Cost
```

Check your understanding

- Define freshness SLO.
- Define RPO/RTO.
- Describe a replay plan.
- Name five operational metrics.

Study prompt: explain the concept aloud, modify the example, test an edge case, and record one common failure mode.